

Informe de laboratorio 05

Tema: Árbol de Expansión Mínimo: Kruskal, Prim y Boruvka

Nota

Estudiante	Escuela	Asignatura
Capatinta Almanza Camilo B. A. ccapatinta@unsa.edu.pe	Escuela Profesional de Ingeniería de Sistemas	Lab. Análisis y Diseño de Algoritmos Semestre: IV Código: 1702231

Laboratorio	Tema	Duración
05	Árbol de Expansión Mínimo: Kruskal, Prim y Boruvka	02 horas

Semestre académico	Fecha de inicio	Fecha de entrega
2026 - B	01/10/2026	-/10/2026

1. Ejercicios/Problemas propuestos

1.1. Ejercicio 1

/relleno

```

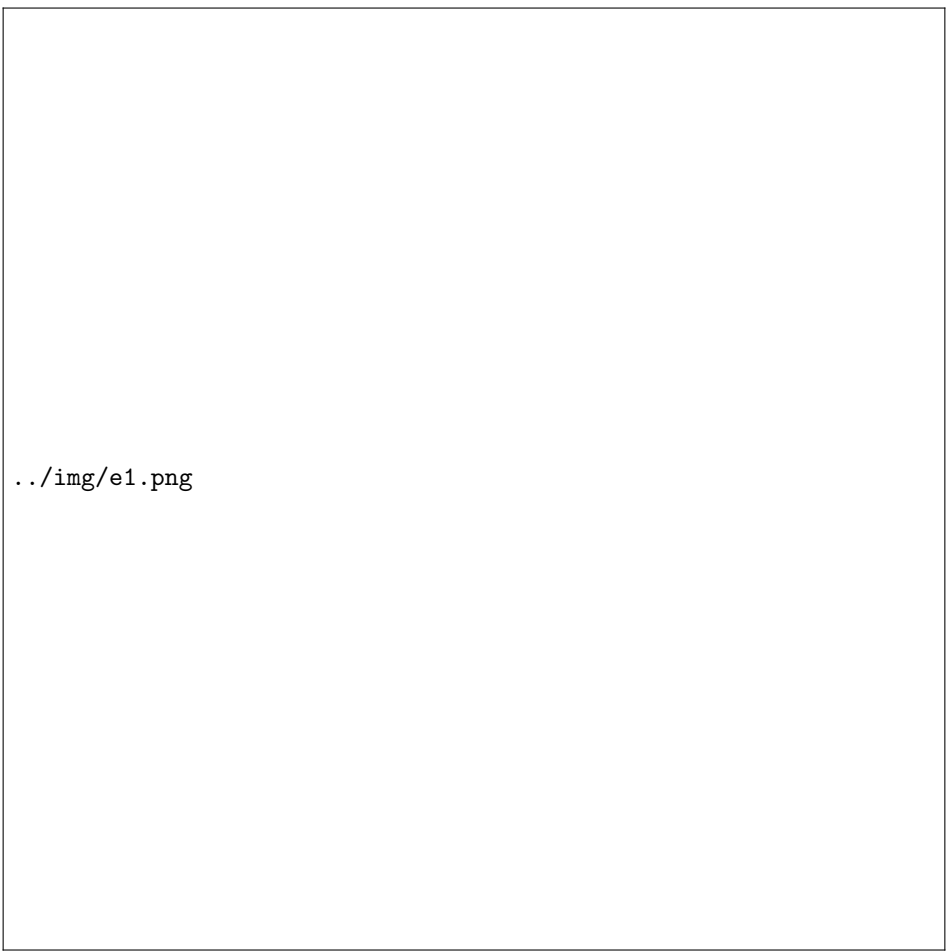
1
2 #include <bits/stdc++.h>
3 using namespace std;
4
5 struct Arista {
6     int u, v;
7     long long w;
8 };
9
10 struct UnionFind {
11     vector<int> padre;
12     vector<int> rango;
13     int componentes;
14
15     UnionFind(int n) : padre(n), rango(n, 0), componentes(n) {
16         iota(padre.begin(), padre.end(), 0);
    }

```

```
17 }
18
19 int buscar(int x) {
20     if (padre[x] != x)
21         padre[x] = buscar(padre[x]);
22     return padre[x];
23 }
24
25 bool unir(int a, int b) {
26     a = buscar(a);
27     b = buscar(b);
28     if (a == b) return false;
29     if (rango[a] < rango[b]) swap(a, b);
30     padre[b] = a;
31     if (rango[a] == rango[b]) rango[a]++;
32     componentes--;
33     return true;
34 }
35 };
36
37 const char* EJEMPLO =
38     "6\n"
39     "Rectorado Biblioteca Laboratorio Cafeteria Gimnasio Auditorio\n"
40     "9\n"
41     "Rectorado Biblioteca 4\n"
42     "Rectorado Cafeteria 8\n"
43     "Biblioteca Laboratorio 8\n"
44     "Biblioteca Cafeteria 11\n"
45     "Laboratorio Gimnasio 7\n"
46     "Cafeteria Gimnasio 1\n"
47     "Cafeteria Auditorio 2\n"
48     "Gimnasio Auditorio 6\n"
49     "Laboratorio Auditorio 4\n";
50
51 int main() {
52
53     cout << "Usar datos de ejemplo? (s/n): ";
54     char op; cin >> op;
55     istreamstream ejemplo(EJEMPLO);
56     istream& in = (op == 's' || op == 'S') ? static_cast<istream&>(ejemplo) : cin;
57     if (op != 's' && op != 'S')
58         cout << "Ingrese n, luego los n nombres, luego m y luego m lineas 'A B costo':\n";
59
60     int n, m;
61     in >> n;
62     vector<string> nombre(n);
63     unordered_map<string, int> id;
64     for (int i = 0; i < n; i++) { in >> nombre[i]; id[nombre[i]] = i; }
65
66     in >> m;
67     vector<Arista> aristas;
68     for (int i = 0; i < m; i++) {
69         string a, b; long long c;
70         in >> a >> b >> c;
71         if (!id.count(a) || !id.count(b) || c < 0) {
72             cout << "Tendido invalido ignorado: " << a << " " << b << " " << c << "\n";
```

```
73         continue;
74     }
75     if (id[a] == id[b]) continue;
76     aristas.push_back({id[a], id[b], c});
77 }
78
79 sort(aristas.begin(), aristas.end(),
80      [](const Arista& x, const Arista& y) { return x.w < y.w; });
81
82 UnionFind uf(n);
83 vector<Arista> elegidas;
84 long long costoTotal = 0;
85
86 cout << "\n--- Recorrido de Kruskal (aristas por costo creciente) ---\n";
87 for (const Arista& e : aristas) {
88     if (uf.unir(e.u, e.v)) {
89         elegidas.push_back(e);
90         costoTotal += e.w;
91         cout << "ACEPTADA " << nombre[e.u] << " - " << nombre[e.v]
92              << " (costo " << e.w << ")\n";
93         if ((int)elegidas.size() == n - 1) break;
94     } else {
95         cout << "rechazada " << nombre[e.u] << " - " << nombre[e.v]
96              << " (costo " << e.w << " ) -> formaria un ciclo\n";
97     }
98 }
99
100 cout << "\n=== RED DE CABLEADO DE MENOR COSTO ===\n";
101 for (const Arista& e : elegidas)
102     cout << nombre[e.u] << " <-> " << nombre[e.v] << " costo " << e.w << "\n";
103 cout << "Costo total: " << costoTotal << "\n";
104
105 if (uf.componentes > 1)
106     cout << "AVISO: los tendidos dados no conectan todos los edificios; "
107          << "quedan " << uf.componentes << " grupos separados.\n";
108 return 0;
109 }
```

■ /relleno



../img/e1.png

Figura 1: Ejecución del ejercicio 1

1.2. Ejercicio 2

/relleno

```
1
2 #include <bits/stdc++.h>
3 using namespace std;
4
5 struct Arista { int u, v; long long w; };
6
7 bool menor(const Arista& a, const Arista& b) {
8     if (a.w != b.w) return a.w < b.w;
9     if (a.u != b.u) return a.u < b.u;
10    return a.v < b.v;
11 }
12
13 struct UnionFind {
14     vector<int> padre, rango;
15     UnionFind(int n) : padre(n), rango(n, 0) { iota(padre.begin(), padre.end(), 0); }
16     int buscar(int x) { return padre[x] == x ? x : padre[x] = buscar(padre[x]); }
```

```
17     bool unir(int a, int b) {
18         a = buscar(a); b = buscar(b);
19         if (a == b) return false;
20         if (rango[a] < rango[b]) swap(a, b);
21         padre[b] = a;
22         if (rango[a] == rango[b]) rango[a]++;
23         return true;
24     }
25 };
26
27 vector<Arista> kruskal(int n, vector<Arista> aristas) {
28     sort(aristas.begin(), aristas.end(), menor);
29     UnionFind uf(n);
30     vector<Arista> res;
31     for (const Arista& e : aristas)
32         if (uf.unir(e.u, e.v)) res.push_back(e);
33     return res;
34 }
35
36 struct Candidato {
37     Arista e;
38     int destino;
39 };
40
41 struct MayorPrioridad {
42     bool operator()(const Candidato& a, const Candidato& b) const { return menor(b.e, a.e); }
43 };
44
45 vector<Arista> prim(int n, const vector<vector<Candidato>>& ady) {
46     vector<bool> visitado(n, false);
47     vector<Arista> res;
48     priority_queue<Candidato, vector<Candidato>, MayorPrioridad> pq;
49
50     for (int inicio = 0; inicio < n; inicio++) {
51         if (visitado[inicio]) continue;
52         visitado[inicio] = true;
53         for (const Candidato& c : ady[inicio]) pq.push(c);
54
55         while (!pq.empty()) {
56             Candidato c = pq.top(); pq.pop();
57             if (visitado[c.destino]) continue;
58             visitado[c.destino] = true;
59             res.push_back(c.e);
60             for (const Candidato& sig : ady[c.destino])
61                 if (!visitado[sig.destino]) pq.push(sig);
62         }
63     }
64     return res;
65 }
66
67 long long pesoTotal(const vector<Arista>& v) {
68     long long s = 0;
69     for (const Arista& e : v) s += e.w;
70     return s;
71 }
72
```

```

73 void imprimir(const string& titulo, const vector<Arista>& v) {
74     cout << titulo << "\n";
75     for (const Arista& e : v) cout << " " << e.u << " - " << e.v << " (peso " << e.w <<
    ↪ ")\n";
76     cout << " Peso total: " << pesoTotal(v) << "\n";
77 }
78
79 const char* EJEMPLO =
80     "7 11\n"
81     "0 1 7\n0 3 5\n1 2 8\n1 3 9\n1 4 7\n2 4 5\n3 4 15\n3 5 6\n4 5 8\n4 6 9\n5 6 11\n";
82
83 int main() {
84     cout << "Usar grafo de ejemplo? (s/n): ";
85     char op; cin >> op;
86     istringstream ejemplo(EJEMPLO);
87     istream& in = (op == 's' || op == 'S') ? static_cast<istream&>(ejemplo) : cin;
88     if (op != 's' && op != 'S') cout << "Ingrese 'n m' y luego m lineas 'u v w' (vertices
    ↪ 0..n-1):\n";
89
90     int n, m;
91     in >> n >> m;
92     vector<Arista> aristas;
93     vector<vector<Candidato>> ady(n);
94     for (int i = 0; i < m; i++) {
95         int u, v; long long w;
96         in >> u >> v >> w;
97         if (u == v || u < 0 || v < 0 || u >= n || v >= n) continue;
98         if (u > v) swap(u, v);
99         Arista e{u, v, w};
100         aristas.push_back(e);
101         ady[u].push_back({e, v});
102         ady[v].push_back({e, u});
103     }
104
105     vector<Arista> aK = kruskal(n, aristas);
106     vector<Arista> aP = prim(n, ady);
107
108     imprimir("=== KRUSKAL ===", aK);
109     imprimir("=== PRIM (cola de prioridad) ===", aP);
110
111     vector<Arista> ordK = aK, ordP = aP;
112     sort(ordK.begin(), ordK.end(), menor);
113     sort(ordP.begin(), ordP.end(), menor);
114     bool mismasAristas = ordK.size() == ordP.size() &&
115         equal(ordK.begin(), ordK.end(), ordP.begin(),
116             [](const Arista& a, const Arista& b) { return a.u == b.u && a.v == b.v && a.w
    ↪ == b.w; });
117
118     cout << "\n=== COMPARACION ===\n";
119     cout << "Peso Kruskal = " << pesoTotal(aK) << " | Peso Prim = " << pesoTotal(aP)
120         << (pesoTotal(aK) == pesoTotal(aP) ? " -> IGUALES\n" : " -> DIFERENTES
    ↪ (error)\n");
121     cout << "Aristas: " << (mismasAristas ? "ambos arboles tienen EXACTAMENTE las mismas
    ↪ aristas\n"
122                                     : "los conjuntos de aristas difieren\n");
123     cout << "Orden de seleccion: Kruskal es global (por peso), Prim crece desde un
    ↪ vertice;\n";

```

```
124     << "el orden de aparicion difiere pero el arbol final es el mismo.\n";
125     return 0;
126 }
```

■ /relleno

../img/e2.png

Figura 2: Ejecución del ejercicio 2

1.3. Ejercicio 3

/relleno

```
1
2 #include <bits/stdc++.h>
3 using namespace std;
4
5 struct Arista { int u, v; long long w; };
6
7 bool menor(const Arista& a, const Arista& b) {
8     if (a.w != b.w) return a.w < b.w;
```

```
9     if (a.u != b.u) return a.u < b.u;
10    return a.v < b.v;
11 }
12
13 struct UnionFind {
14     vector<int> padre, rango;
15     int componentes;
16     UnionFind(int n) : padre(n), rango(n, 0), componentes(n) { iota(padre.begin(),
17     ↪ padre.end(), 0); }
18     int buscar(int x) { return padre[x] == x ? x : padre[x] = buscar(padre[x]); }
19     bool unir(int a, int b) {
20         a = buscar(a); b = buscar(b);
21         if (a == b) return false;
22         if (rango[a] < rango[b]) swap(a, b);
23         padre[b] = a;
24         if (rango[a] == rango[b]) rango[a]++;
25         componentes--;
26         return true;
27     }
28 };
29
30 long long pesoTotal(const vector<Arista>& v) {
31     long long s = 0;
32     for (auto& e : v) s += e.w;
33     return s;
34 }
35
36 vector<Arista> boruvka(int n, const vector<Arista>& E) {
37     UnionFind uf(n);
38     vector<Arista> res;
39     int ronda = 0;
40
41     cout << "Componentes iniciales: " << uf.componentes << "\n";
42     while (uf.componentes > 1) {
43         ronda++;
44
45         vector<int> mejor(n, -1);
46         for (int i = 0; i < (int)E.size(); i++) {
47             int a = uf.buscar(E[i].u), b = uf.buscar(E[i].v);
48             if (a == b) continue;
49             if (mejor[a] == -1 || menor(E[i], E[mejor[a]])) mejor[a] = i;
50             if (mejor[b] == -1 || menor(E[i], E[mejor[b]])) mejor[b] = i;
51         }
52
53         cout << "\n--- Ronda " << ronda << " ---\n";
54         int agregadas = 0;
55         for (int r = 0; r < n; r++) {
56             if (mejor[r] == -1) continue;
57             const Arista& e = E[mejor[r]];
58             if (uf.unir(e.u, e.v)) {
59                 res.push_back(e);
60                 agregadas++;
61                 cout << " + arista " << e.u << " - " << e.v << " (peso " << e.w << ") \n";
62             }
63         }
64     }
```



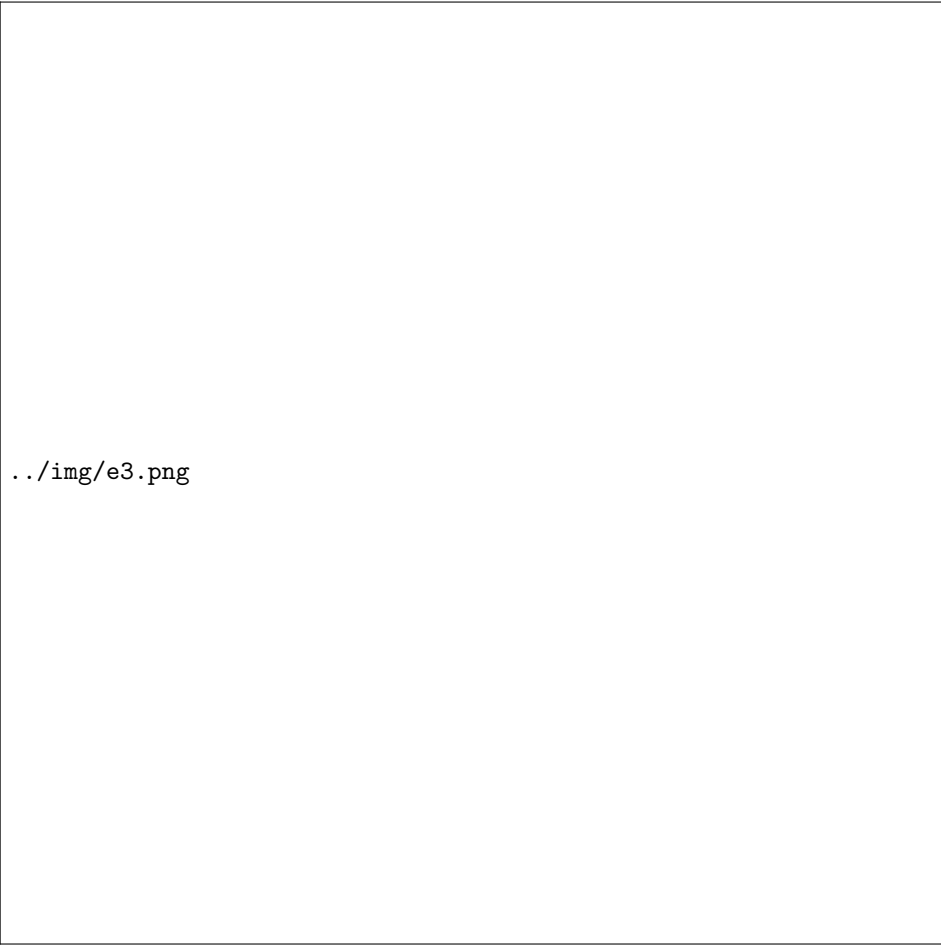
```

64     cout << "  Aristas agregadas: " << agregadas
65         << " | Componentes restantes: " << uf.componentes << "\n";
66
67     if (agregadas == 0) {
68         cout << "  (No hay mas aristas salientes: el grafo no es conexo)\n";
69         break;
70     }
71 }
72 return res;
73 }
74
75 vector<Arista> kruskal(int n, vector<Arista> E) {
76     sort(E.begin(), E.end(), menor);
77     UnionFind uf(n);
78     vector<Arista> res;
79     for (auto& e : E) if (uf.unir(e.u, e.v)) res.push_back(e);
80     return res;
81 }
82
83 vector<Arista> prim(int n, const vector<Arista>& E) {
84
85     vector<vector<pair<int,int>>> ady(n);
86     for (int i = 0; i < (int)E.size(); i++) {
87         ady[E[i].u].push_back({i, E[i].v});
88         ady[E[i].v].push_back({i, E[i].u});
89     }
90
91     auto cmp = [&](const pair<int,int>& a, const pair<int,int>& b) { return
92         ↪ menor(E[b.first], E[a.first]); };
93     priority_queue<pair<int,int>, vector<pair<int,int>>, decltype(cmp)> pq(cmp);
94
95     vector<bool> vis(n, false);
96     vector<Arista> res;
97     for (int s = 0; s < n; s++) {
98         if (vis[s]) continue;
99         vis[s] = true;
100        for (auto& p : ady[s]) pq.push(p);
101        while (!pq.empty()) {
102            auto [idx, dest] = pq.top(); pq.pop();
103            if (vis[dest]) continue;
104            vis[dest] = true;
105            res.push_back(E[idx]);
106            for (auto& p : ady[dest]) if (!vis[p.second]) pq.push(p);
107        }
108    }
109    return res;
110 }
111 bool iguales(vector<Arista> a, vector<Arista> b) {
112     if (a.size() != b.size()) return false;
113     sort(a.begin(), a.end(), menor);
114     sort(b.begin(), b.end(), menor);
115     for (size_t i = 0; i < a.size(); i++)
116         if (a[i].u != b[i].u || a[i].v != b[i].v || a[i].w != b[i].w) return false;
117     return true;
118 }

```

```
119
120 const char* EJEMPLO =
121     "8 13\n"
122     "0 1 4\n0 2 3\n1 2 5\n1 3 6\n2 3 7\n2 4 8\n3 4 2\n3 5 9\n4 5 10\n4 6 11\n5 6 1\n5 7 12\n6
    ↪ 7 13\n";
123
124 int main() {
125     cout << "Usar grafo de ejemplo? (s/n): ";
126     char op; cin >> op;
127     istringstream ejemplo(EJEMPLO);
128     istream& in = (op == 's' || op == 'S') ? static_cast<istream&>(ejemplo) : cin;
129     if (op != 's' && op != 'S') cout << "Ingrese 'n m' y luego m lineas 'u v w':\n";
130
131     int n, m;
132     in >> n >> m;
133     vector<Arista> E;
134     for (int i = 0; i < m; i++) {
135         int u, v; long long w;
136         in >> u >> v >> w;
137         if (u == v || u < 0 || v < 0 || u >= n || v >= n) continue;
138         if (u > v) swap(u, v);
139         E.push_back({u, v, w});
140     }
141
142     vector<Arista> aB = boruvka(n, E);
143     vector<Arista> aK = kruskal(n, E);
144     vector<Arista> aP = prim(n, E);
145
146     cout << "\n=== VERIFICACION FINAL ===\n";
147     cout << "Peso Boruvka = " << pesoTotal(aB) << "\n";
148     cout << "Peso Kruskal = " << pesoTotal(aK) << "\n";
149     cout << "Peso Prim = " << pesoTotal(aP) << "\n";
150     cout << "Boruvka == Kruskal (aristas): " << (iguales(aB, aK) ? "SI" : "NO") << "\n";
151     cout << "Boruvka == Prim (aristas): " << (iguales(aB, aP) ? "SI" : "NO") << "\n";
152     return 0;
153 }
```

■ /relleno



../img/e3.png

Figura 3: Ejecución del ejercicio 3

1.4. Ejercicio 4

/relleno

```
1
2 #include <bits/stdc++.h>
3 using namespace std;
4 using Reloj = chrono::steady_clock;
5
6 struct Arista { int u, v; int w; };
7
8 bool menor(const Arista& a, const Arista& b) {
9     if (a.w != b.w) return a.w < b.w;
10    if (a.u != b.u) return a.u < b.u;
11    return a.v < b.v;
12 }
13
14 vector<Arista> generarGrafo(int n, long long m, mt19937& rng) {
15     vector<Arista> E;
16     E.reserve(m);
```

```
17 vector<char> usada((size_t)n * n, 0);
18 uniform_int_distribution<int> peso(1, 1000000);
19
20 auto agregar = [&](int u, int v) {
21     if (u > v) swap(u, v);
22     usada[(size_t)u * n + v] = 1;
23     E.push_back({u, v, peso(rng)});
24 };
25
26 vector<int> perm(n);
27 iota(perm.begin(), perm.end(), 0);
28 shuffle(perm.begin(), perm.end(), rng);
29 for (int i = 1; i < n; i++) {
30     int j = uniform_int_distribution<int>(0, i - 1)(rng);
31     agregar(perm[i], perm[j]);
32 }
33
34 uniform_int_distribution<int> vert(0, n - 1);
35 while ((long long)E.size() < m) {
36     int u = vert(rng), v = vert(rng);
37     if (u == v) continue;
38     if (u > v) swap(u, v);
39     if (usada[(size_t)u * n + v]) continue;
40     agregar(u, v);
41 }
42 return E;
43 }
44
45 struct UnionFind {
46     vector<int> padre, rango;
47     UnionFind(int n) : padre(n), rango(n, 0) { iota(padre.begin(), padre.end(), 0); }
48     int buscar(int x) {
49         int r = x;
50         while (padre[r] != r) r = padre[r];
51         while (padre[x] != r) { int sig = padre[x]; padre[x] = r; x = sig; }
52         return r;
53     }
54     bool unir(int a, int b) {
55         a = buscar(a); b = buscar(b);
56         if (a == b) return false;
57         if (rango[a] < rango[b]) swap(a, b);
58         padre[b] = a;
59         if (rango[a] == rango[b]) rango[a]++;
60         return true;
61     }
62 };
63
64 long long kruskal(int n, vector<Arista> E) {
65     sort(E.begin(), E.end(), menor);
66     UnionFind uf(n);
67     long long total = 0;
68     int cnt = 0;
69     for (const Arista& e : E) {
70         if (uf.unir(e.u, e.v)) {
71             total += e.w;
72             if (++cnt == n - 1) break;
```

```
73     }
74 }
75 return total;
76 }
77
78 long long primCola(int n, const vector<Arista>& E) {
79
80     vector<vector<pair<int,int>>> ady(n);
81     for (const Arista& e : E) {
82         ady[e.u].push_back({e.v, e.w});
83         ady[e.v].push_back({e.u, e.w});
84     }
85     using Par = pair<int,int>;
86     priority_queue<Par, vector<Par>, greater<Par>> pq;
87     vector<char> vis(n, 0);
88     long long total = 0;
89     int cnt = 0;
90
91     pq.push({0, 0});
92     while (!pq.empty() && cnt < n) {
93         auto [w, u] = pq.top(); pq.pop();
94         if (vis[u]) continue;
95         vis[u] = 1; total += w; cnt++;
96         for (auto [v, pw] : ady[u])
97             if (!vis[v]) pq.push({pw, v});
98     }
99     return total;
100 }
101
102 long long primMatriz(int n, const vector<Arista>& E) {
103     const int INF = INT_MAX;
104     vector<int> mat((size_t)n * n, INF);
105     for (const Arista& e : E) {
106         mat[(size_t)e.u * n + e.v] = e.w;
107         mat[(size_t)e.v * n + e.u] = e.w;
108     }
109     vector<int> dist(n, INF);
110     vector<char> vis(n, 0);
111     long long total = 0;
112     dist[0] = 0;
113     for (int it = 0; it < n; it++) {
114         int u = -1;
115         for (int v = 0; v < n; v++)
116             if (!vis[v] && (u == -1 || dist[v] < dist[u])) u = v;
117         vis[u] = 1;
118         total += dist[u];
119         const int* fila = &mat[(size_t)u * n];
120         for (int v = 0; v < n; v++)
121             if (!vis[v] && fila[v] < dist[v]) dist[v] = fila[v];
122     }
123     return total;
124 }
125
126 template <class F>
127 double medir(F f, int reps, long long& resultado) {
128     double acum = 0;
```

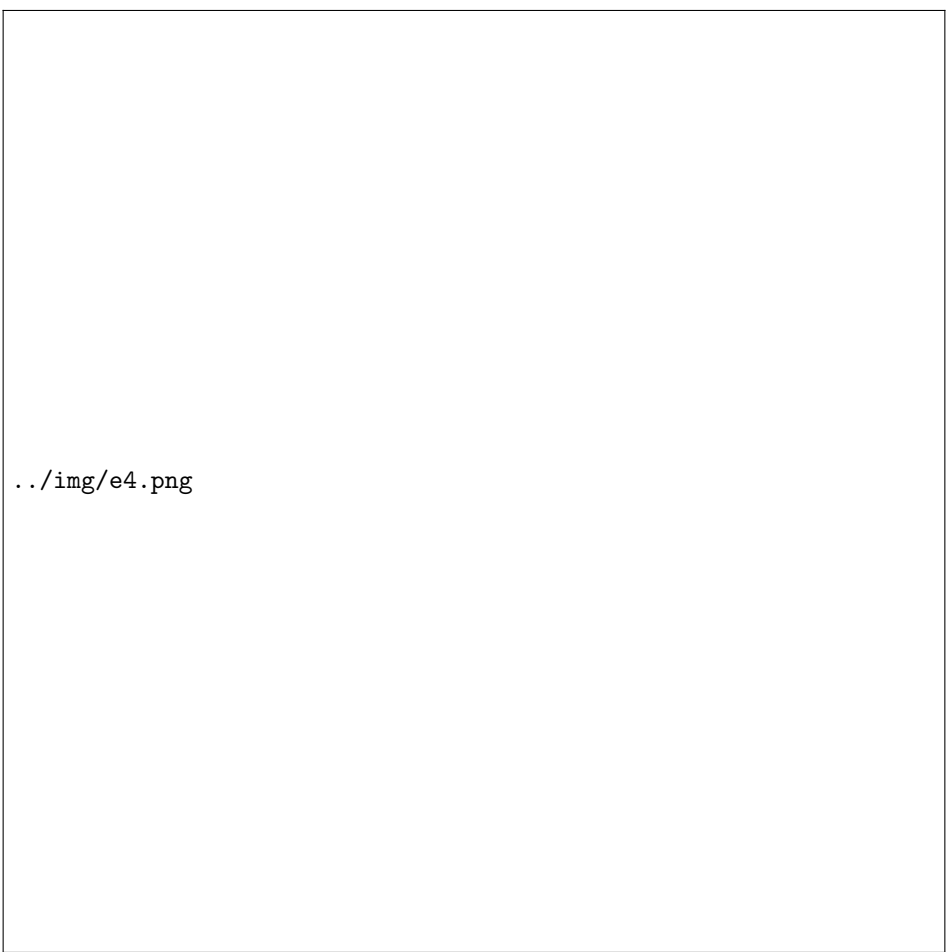
```

129     for (int i = 0; i < reps; i++) {
130         auto t0 = Reloj::now();
131         resultado = f();
132         auto t1 = Reloj::now();
133         acum += chrono::duration<double, milli>(t1 - t0).count();
134     }
135     return acum / reps;
136 }
137
138 int main() {
139     mt19937 rng(12345);
140     const vector<int> tamanos = {100, 500, 1000, 2000};
141     const int REPS = 3;
142
143     cout << fixed << setprecision(3);
144     cout << left << setw(9) << "Tipo" << right << setw(7) << "n" << setw(10) << "m"
145         << setw(13) << "Kruskal(ms)" << setw(13) << "PrimCola(ms)" << setw(14) <<
146         << "PrimMatriz(ms)"
147         << " Mas rapido Pesos iguales\n";
148     cout << string(88, '-') << "\n";
149
150     vector<string> resumen;
151     for (string tipo : {"Disperso", "Denso"}) {
152         for (int n : tamanos) {
153             long long maxAristas = (long long)n * (n - 1) / 2;
154             long long m = (tipo == "Disperso") ? min(maxAristas, 3LL * n)
155                 : (long long)(0.7 * maxAristas);
156             vector<Arista> E = generarGrafo(n, m, rng);
157
158             long long tk, tp, tm;
159             double msK = medir([&] { return kruskal(n, E); }, REPS, tk);
160             double msP = medir([&] { return primCola(n, E); }, REPS, tp);
161             double msM = medir([&] { return primMatriz(n, E); }, REPS, tm);
162
163             string ganador = "Kruskal";
164             double mejor = msK;
165             if (msP < mejor) { mejor = msP; ganador = "PrimCola"; }
166             if (msM < mejor) { mejor = msM; ganador = "PrimMatriz"; }
167             bool ok = (tk == tp && tp == tm);
168
169             cout << left << setw(9) << tipo << right << setw(7) << n << setw(10) << E.size()
170                 << setw(13) << msK << setw(13) << msP << setw(14) << msM
171                 << " " << left << setw(12) << ganador << (ok ? "SI" : "NO (error)") <<
172                 << "\n";
173         }
174     }
175     cout << string(88, '-') << "\n";
176
177     cout << "\nCONCLUSIONES TEORICAS (contrastar con la tabla):\n"
178         << " - Grafo disperso ( $E \sim V$ ): Kruskal  $O(E \log E)$  y Prim-cola  $O(E \log V)$  son
179         << "     equivalentes;\n"
180         << "     Prim-matriz pierde por pagar  $O(V^2)$  y memoria  $V^2$ .\n"
181         << " - Grafo denso ( $E \sim V^2$ ): Prim-matriz  $O(V^2)$  supera a los que usan log,
182         << "     pues\n"
183         << "     Kruskal ordena  $\sim V^2$  aristas ( $V^2 \log V$ ) y Prim-cola maneja un heap
184         << "     enorme.\n";

```

```
180     return 0;
181 }
```

■ /relleno



../img/e4.png

Figura 4: Ejecución del ejercicio 4

1.5. Ejercicio 5

/relleno

```
1
2 #include <bits/stdc++.h>
3 using namespace std;
4
5 struct Tramo {
6     int u, v;
7     double km;
8     double costo;
9 };
```

```
10
11 struct UnionFind {
12     vector<int> padre, rango;
13     int componentes;
14     UnionFind(int n) : padre(n), rango(n, 0), componentes(n) { iota(padre.begin(),
15         ↪ padre.end(), 0); }
16     int buscar(int x) { return padre[x] == x ? x : padre[x] = buscar(padre[x]); }
17     bool unir(int a, int b) {
18         a = buscar(a); b = buscar(b);
19         if (a == b) return false;
20         if (rango[a] < rango[b]) swap(a, b);
21         padre[b] = a;
22         if (rango[a] == rango[b]) rango[a]++;
23         componentes--;
24         return true;
25     }
26 };
27
28 const char* EJEMPLO =
29     "8\n"
30     "Yura Cayma Uchumayo Characato Sabandia Socabaya Chiguata Polobaya\n"
31     "14\n"
32     "Yura Cayma 12 60\n"
33     "Yura Uchumayo 18 75\n"
34     "Cayma Uchumayo 15 50\n"
35     "Cayma Socabaya 10 45\n"
36     "Uchumayo Socabaya 9 55\n"
37     "Socabaya Characato 8 40\n"
38     "Socabaya Sabandia 7 40\n"
39     "Characato Sabandia 5 35\n"
40     "Characato Chiguata 14 70\n"
41     "Sabandia Chiguata 17 80\n"
42     "Sabandia Polobaya 20 85\n"
43     "Chiguata Polobaya 16 90\n"
44     "Uchumayo Polobaya 30 95\n"
45     "Cayma Characato 13 50\n";
46
47 int main() {
48     cout << "Usar red de ejemplo? (s/n): ";
49     char op; cin >> op;
50     istringstream ejemplo(EJEMPLO);
51     istream& in = (op == 's' || op == 'S') ? static_cast<istream&>(ejemplo) : cin;
52     if (op != 's' && op != 'S')
53         cout << "Ingrese n, los nombres, m y luego m lineas 'A B km costo_por_km':\n";
54
55     int n, m;
56     in >> n;
57     vector<string> nombre(n);
58     unordered_map<string, int> id;
59     for (int i = 0; i < n; i++) { in >> nombre[i]; id[nombre[i]] = i; }
60
61     in >> m;
62     vector<Tramo> tramos;
63     double costoTodo = 0;
64     for (int i = 0; i < m; i++) {
65         string a, b; double km, cpk;
```



```

65     in >> a >> b >> km >> cpk;
66     if (!id.count(a) || !id.count(b) || km <= 0 || cpk < 0 || a == b) {
67         cout << "Tramo invalido ignorado: " << a << " " << b << "\n";
68         continue;
69     }
70     double costo = km * cpk;
71     tramos.push_back({id[a], id[b], km, costo});
72     costoTodo += costo;
73 }
74
75 sort(tramos.begin(), tramos.end(), [](const Tramo& x, const Tramo& y) {
76     if (x.costo != y.costo) return x.costo < y.costo;
77     if (min(x.u, x.v) != min(y.u, y.v)) return min(x.u, x.v) < min(y.u, y.v);
78     return max(x.u, x.v) < max(y.u, y.v);
79 });
80
81 UnionFind uf(n);
82 vector<Tramo> red;
83 double costoMin = 0, kmTotal = 0;
84 for (const Tramo& t : tramos) {
85     if (uf.unir(t.u, t.v)) {
86         red.push_back(t);
87         costoMin += t.costo;
88         kmTotal += t.km;
89         if ((int)red.size() == n - 1) break;
90     }
91 }
92
93 cout << fixed << setprecision(1);
94 cout << "\n=== RED DE DISTRIBUCION DE COSTO MINIMO ===\n";
95 for (const Tramo& t : red)
96     cout << setw(11) << nombre[t.u] << " <-> " << left << setw(11) << nombre[t.v] <<
97         << setw(6) << t.km << " km    costo " << setw(8) << t.costo << " (miles de
98         << S/)\n";
99
100 cout << "\nTramos contruidos : " << red.size() << " de " << tramos.size() << "
101     << posibles\n";
102 cout << "Longitud total      : " << kmTotal << " km\n";
103 cout << "COSTO MINIMO TOTAL : " << costoMin << " (miles de S/)\n";
104
105 if (uf.componentes > 1) {
106     cout << "\nAVISO: la red NO conecta todas las localidades; quedan " << uf.componentes
107         << " zonas aisladas. Se necesitan tramos adicionales.\n";
108 } else {
109     cout << "Costo de construir TODOS los tramos: " << costoTodo << " (miles de S/)\n";
110     cout << "Ahorro frente a construirlos todos : " << costoTodo - costoMin << " ("
111         << 100.0 * (costoTodo - costoMin) / costoTodo << "%)\n";
112 }
113 return 0;
114 }

```

■ /relleno



../img/e5.png

Figura 5: Ejecución del ejercicio 5

2. Cuestionario

1. ¿Por qué Kruskal necesita una estructura Union-Find, y qué problema resuelve la compresión de caminos dentro de esa estructura?
2. ¿En qué se diferencia la estrategia de Prim de la de Kruskal, si ambos son algoritmos golosos?
3. ¿Puede un grafo tener más de un árbol de expansión mínimo? ¿Bajo qué condición ocurre esto?
4. ¿Por qué se dice que Boruvka es un algoritmo naturalmente paralelizable?
5. ¿Qué diferencia fundamental existe entre el problema del árbol de expansión mínimo y el problema del camino más corto visto en el Laboratorio 4?

3. Conclusiones